

Giving Agents Memory

LLMs Do Not Have Memory

When we are building an Agent, we need it to remember its actions and the result of those actions. For example, if it tries to create a calendar event for a meeting and the API call fails due to an incorrect parameter value that it provided, we want it to remember that the API call failed and why. This way, it can correct the mistake and try again. If we have a complex task that we break down into multiple steps, we need the Agent to remember the results of each step to ensure that it can continue the task from where it left off. Memory is crucial for Agents.

LLMs Do Not Have Memory

When interacting with an LLM, **the model does not inherently “remember” previous conversations or responses**. Every time you call the model, it generates a response based solely on the information provided in the messages parameter. If previous context is not included in the messages, the model will not have any knowledge of it.

This means that to simulate continuity in a conversation, you must explicitly pass all relevant prior messages (including system, user, and assistant roles) in the messages list for each request.

Example 1: Missing Context in the Prompt

```
messages = [
    {"role": "system", "content": "You are an expert software engineer that  
prefers functional programming.-"},
    {"role": "user", "content": "Write a function to swap the keys and values  
in a dictionary."}
]

response = generate_response(messages)
print(response)

# Second query without including the previous response
messages = [
    {"role": "user", "content": "Update the function to include  
documentation."}
]

response = generate_response(messages)
print(response)
```

Output:

1. For the first request:

```
def swap_keys_values(d):
    return {v: k for k, v in d.items()}
```

2. For the second request:

I'm not sure what function you're referring to. Can you clarify?

Explanation: In the second request, the model doesn't “remember” the function it wrote in the first interaction. Since the information is not included in the second prompt, the model cannot connect the two.

Example 2: Including Previous Responses for Continuity

To fix this issue, we need to add new messages with the "assistant" role to the messages list with the content of the prior response from the LLM. This way, the model can see what code it wrote previously and can build on that.

```

messages = {
    ("role": "system", "content": "You are an expert software engineer that  
prefers functional programming."),
    ("role": "user", "content": "Write a function to swap the keys and values  
in a dictionary.")
}

response = generate_response(messages)

print(response)

# We are going to make this verbose so it is clear what
# is going on. In a real application, you would likely
# just append to the messages list.
messages = [
    ("role": "system", "content": "You are an expert software engineer that  
prefers functional programming."),
    ("role": "user", "content": "Write a function to swap the keys and values  
in a dictionary."),
    # Here is the assistant's response from the previous step
    # with the code. This gives it "memory" of the previous
    # interaction.
    ("role": "assistant", "content": response),

    # Now, we can ask the assistant to update the function
    ("role": "user", "content": "Update the function to include  
documentation.")
]

response = generate_response(messages)

print(response)

```

Output:

```
def swap_keys_values(d):
    """
    Swaps the keys and values in the given dictionary.

    Args:
        d (dict): The input dictionary with unique values.

    Returns:
        dict: A new dictionary where the keys and values are swapped.

    Raises:
        ValueError: If the dictionary contains duplicate values, as they
            cannot be used as unique keys in the result.

    Example:
        >>> swap_keys_values({'a': 1, 'b': 2, 'c': 3})
        {'1': 'a', 2: 'b', 3: 'c'}
        >>> swap_keys_values({'x': 10, 'y': 20})
        {'10': 'x', 20: 'y'}

    """
    if len(d.values()) != len(set(d.values())):
        raise ValueError("Duplicate values detected. Cannot swap keys and
            values.")
    return {v: k for k, v in d.items()}
```

Explanation: By including the assistant's previous response in the messages, the model can maintain context and provide an appropriate response to the follow-up question.

Key Takeaways

1. **No Inherent Memory:** The LLM has no knowledge of past interactions unless explicitly provided in the current prompt (via messages).
2. **Provide Full Context:** To simulate continuity in a conversation, include all relevant messages (both user and assistant responses) in the messages parameter.
3. **Role of Assistant Messages:** Adding previous responses as assistant messages allows the model to maintain a coherent conversation and build on earlier exchanges. For an agent, this will allow it to remember what actions, such as API calls, it took in the past.
4. **Memory Management:** We can control what the LLM remembers or does not remember by managing what messages go into the conversation. Causing the LLM to forget things can be a powerful tool in some circumstances, such as when we need to break a pattern of poor responses from an Agent.

Why This Matters

Understanding the stateless nature of LLMs is crucial for designing agents that rely on multi-turn conversations with their environment. Developers must explicitly manage and provide context to ensure the model generates accurate and relevant responses.

Expand

✓ Completed

Go to next item